

IG03 Final Project Report

PEDRO NASCIMENTO COÊLHO, Institut Polytechnique de Paris, France



1 INTRODUCTION

In this project, the goal was to develop a 3D model program, exploring 3 modules from the graphic processing domain: Geometry, Rendering and Simulation. The chosen approach was to create in OpenGL a generator of custom graphical and physical objects, which can have different shapes, colors, textures and can present rigid body behaviour.

For Geometry, different options of meshes were made available, including an option to generate a 3D voxelized model from a 2D binary shape. Besides, a subdivision option was implemented to smooth shapes. For Rendering, the project worked with specific techniques for textures, lighting and shading. The focus of the project was in the **Simulation** part, where rigid bodies were built and a robust collision solver was developed, which is able to deal with interactions between objects with any custom shape.

2 RELATED WORKS

This project was heavily inspired by game engines, such as Unity [9], Godot [3] and Unreal [2], where a user can easily create and animate 3D models. For creating shapes, a convenient approach was to use voxels, the pixel equivalent for 3D rendering, an old geometry

technique popularized by games like Minecraft [6]. One interesting algorithm to render voxels in an efficient way is through Greedy Meshing [5], in which voxels are merged into bigger blocks, in order to achieve the minimum amount of blocks rendered on the image, thus decreasing the amount of vertices to be used on the mesh. There are also many algorithms to smooth shapes which consists of subdivision steps, as the popular Charles Loop Subdivision [4], which was used in the Geometry subsection. Another motivation for this project was to simulate realistic physical interactions between objects applying the theoretical equations of rigid body dynamics, summarized in [11]. Physics simulation is still a challenge for animation, especially due to need of adapting the continuous space and time from the real world to the discrete space and time of the digital world, although some engines are praised for their accurate simulations, such as the Source [10] engine.

3 MODULES DEVELOPED

3.1 Geometry

All these features were implemented in the `Mesh.h` or `Mesh.cpp` files.

3.1.1 Meshes.

5 types of meshes are supported: plane, box, sphere, loaded and custom type.

- Plane: Is a square shape, which has custom side length.

- **Box:** Is a block shape, which has custom width, length and depth. It is centered on the origin by default, but there is also an option to change its center in order to be used in the custom type mesh. Because flat shading usually looks better than smooth shading for rectangular shapes, the normals stored were the face normals.
- **Sphere:** The sphere is actually a polyhedron with radial symmetry that has n meridians with n vertices each. Increasing the resolution parameter increases n , which makes the shape look closer to a sphere, but also increases the number of triangles rendered. Because smooth shading usually looks better than flat shading for round shapes, the normals stored were the vertex normals.
- **Loaded:** It is possible to load meshes stored as .off files specifying the path of the file.
- **Custom type:** Given a binary “draw” stored in a 2D matrix, it creates a 3D model considering each “1” as a voxel, generating boxes that integrate one same mesh. The measures of a voxel can be changed, allowing a great variety of shapes. However, for certain shapes, it wouldn’t be efficient to render a box mesh for each position filled by 1, as many boxes could be rendered together as a single bigger block. That’s why, inspired by the Greedy Meshing algorithm, an algorithm was created to **find bigger rectangles** in a binary matrix, thus creating a box for each rectangle.
The algorithm works in the following way: For each position in the matrix that has a 1, at each iteration a rectangle is searched, expanding the right and bottom sides by one until both sides cannot expand anymore (which means there are no more adjacent columns or rows completely filled with 1’s with the same size as the last expansion). Then this whole region is filled with 0’s and the matrix positions of the upper left corner and the lower right corner are stored to represent the found rectangle. This process repeats until the last position is reached (filling the whole matrix with 0’s). Finally, boxes are generated with width and length equal to the voxel width and length times the width and length of each rectangle found, are centered based on the center of the rectangle and lastly the whole mesh is re-centered considering the geometric center as the origin in the model space.

3.1.2 Subdivision Surface.

To smooth the surfaces, **Charles Loop Subdivision** was implemented as explained in [4]. This subdivision can be applied to meshes with no repetition of vertices, such as the sphere.off mesh.

3.2 Rendering

All these features were implemented in the fragmentShader.glsl file.

3.2.1 Texture.

Texture can be loaded as image files and mapped on mesh using the UV map, which depends on the mesh. Two kinds of meshes were properly implemented to map textures this way: the box and the sphere meshes.

3.2.2 Lightning.

- **Specular lightning:** A specular effect was added, which can be visualized as a brighter region in an object caused by a single source of light. The specular contribution of the specular lightning to the final radiance (before considering shading and light color) $\vec{s}$ is given by the following equation, where $\vec{v}$ is the normal vector from the fragment to the camera, $\vec{r}$ is the vector from the fragment to the light source reflected by the normal at this point, $\vec{c}$ is the surface color and k and α are material parameters.

$$\vec{s} = k \max(\vec{v} \cdot \vec{r}, 0)^\alpha \vec{c} \quad (1)$$

- **TBN Matrix:** Normal mapping textures are usually colored considering a specific orientation of the mesh, which means the desired illumination effect is not achieved when the mesh is rotated. In order to make it work for any orientation, the normal mapping was calculated on the Tangent space, using a TBN matrix for this transformation, as explained in [1].

3.2.3 Shading.

Shadows are generated by turning into black the color of fragments which have distances to the light source bigger than the closest fragments stored in a texture.

Due to space discretizations caused by the resolution limitation, an **acne** effect was identified: fragments near the ones stored in the texture of closest points to the light source were not included in this texture map, and so were covered by shadows although they should be illuminated. To avoid that, a **bias** was added such that the fragment would only be covered by shadow if the difference between its distance from the shadow map frustum to the light source and the distance of the closest fragment was higher than the bias. Being $\vec{l}$ the unit vector pointing to the light and $\vec{n}_f$ the fragment normal vector, the bias was set according to the following heuristics:

$$bias = \max \left(5 \cdot 10^{-3} \left(1 - \vec{n}_f \cdot \vec{l} \right), 5 \cdot 10^{-3} \right) \quad (2)$$

3.3 Simulation

All these features were implemented in the RigidBody.hpp file.

3.3.1 Rigid Body.

Bodies are objects that change their motion based on physical interactions with other objects (which can be either a body or not). Bodies can be passed to a Rigid Solver which sets the laws of dynamics governing the body movement, and also properly deals with collisions between the body and nearby objects. At each simulation step, the dynamics of each body are recalculated and their motion properties are updated.

3.3.2 Collision Solver.

For each rigid body, the Rigid Solver detects all possible collisions with every other object on the scene, calculates the new contributions to the linear and angular momentum of the body, and finally

updates the body's physical properties. The algorithm for solving collisions receives the (triangular) mesh and the world space matrix of an object and then executes 3 steps: vertex-face collision detection, edge-edge collision detection, and momentum update. For each triangle in the mesh of the collision candidate object, both vertex-face and edge-edge collisions are verified.

- **Vertex-Face collision detection**

Considering that edges and vertices are also part of a face, checking for vertex-face collision also includes checking for vertex-vertex and vertex-edge collisions. For each vertex of the body that has not already been tested (this verification is important because some meshes might store the same vertex multiple times), let $\vec{V}_0, \vec{V}_1, \vec{V}_2$ the vertex points of the candidate object triangle, $\vec{W}_b$ the vertex point of the body and $\vec{X}_b$ the point of the body center, all in world coordinates. The collision is detected if:

$$\begin{aligned} & |\beta_v \vec{n}_t \cdot (\vec{W}_b - \vec{V}_0)| < \delta_v \\ & : \vec{n}_t = (\vec{V}_1 - \vec{V}_0) \times (\vec{V}_2 - \vec{V}_0); \\ & \beta_v = \text{sign}(\vec{n}_t \cdot (\vec{X}_b - \vec{W}_b)) \end{aligned} \quad (3)$$

And if:

$$\begin{aligned} & \vec{n}_t \cdot ((\vec{V}_1 - \vec{V}_0) \times (\vec{P} - \vec{V}_0)) > \epsilon \\ & \wedge \vec{n}_t \cdot ((\vec{V}_2 - \vec{V}_1) \times (\vec{P} - \vec{V}_1)) > \epsilon \\ & \wedge \vec{n}_t \cdot ((\vec{V}_0 - \vec{V}_2) \times (\vec{P} - \vec{V}_2)) > \epsilon \\ & : \vec{P} = \vec{W}_b - (\beta_v \vec{n}_t \cdot (\vec{W}_b - \vec{V}_0)) \beta_v \vec{n}_t \end{aligned} \quad (4)$$

Where δ_v and ϵ are precision parameters experimentally set to 10^{-2} and -10^{-6} . The Equation 3 is the distance from point to plane, so it checks if the vertex is close enough to the face. The β_v is necessary to invert the normal in case it's not pointing outwards the body (for example, in a case the body is colliding with the mesh from the other side). Additionally, Equation 4 purpose is to guarantee that the projection of the vertex in the face plane is a point that is inside the triangle region, and not outside, which only happens if all those cross products generate vectors in the same direction of the normal. If the collision is detected, $\beta_v \vec{n}_t$ is stored as a contact normal and the vertex $\vec{W}_b$ is stored as a contact point.

- **Edge-Edge collision detection**

Once edge-face collision happens when two vertices touch a face, this case is already checked during vertex-face collision, so only edge-edge collision remains to be verified. For each edge $\vec{e}_b$ of the body, whose vertices have not been already tested (this verification is needed to avoid collisions of a body edge that is already colliding with a face), let $\vec{W}_b$ be the position of a vertex in that edge, $\vec{e}_t$ the vector of one of the 3 edges of the candidate mesh triangle and $\vec{V}_t$ the position of one vertex of $\vec{e}_t$. The collision is detected if:

$$\begin{aligned} & |\vec{n}_e \cdot (\vec{W}_b - \vec{V}_t)| < \delta_e \\ & : \vec{n}_e = \frac{\vec{e}_b \times \vec{e}_t}{\|\vec{e}_b \times \vec{e}_t\|} \end{aligned} \quad (5)$$

And if:

$$\begin{aligned} & 0 < t_1 < 1 \wedge 0 < t_2 < 1 \\ & : t_1 = \frac{\vec{n}_p \cdot (\vec{W}_b - \vec{V}_t)}{\vec{n}_p \cdot \vec{e}_t}; \vec{n}_p = \frac{\vec{e}_b \times \vec{n}_e}{\|\vec{e}_b \times \vec{n}_e\|} \\ & (t_2 \vec{e}_b + \vec{W}_b) - (t_1 \vec{e}_t + \vec{V}_t) = (\vec{n}_e \cdot (\vec{W}_b - \vec{V}_t)) \vec{n}_e \end{aligned} \quad (6)$$

Where δ_e is a precision parameter experimentally set to $5 \cdot 10^{-3}$. Equation 5 is the distance between the infinite lines that each edge belongs to, so it checks if the edges are in close parallel planes. In Equation 6, the parameter t_1 defines the point of intersection between the infinite lines of the edges projected on the same plane, which comes from the "ray intersection with plane" equation commonly used for ray tracing calculation [7]. More specifically, t_1 is the ratio between the length from $\vec{V}_t$ to the intersection point of the projections and the length of the mesh edge, while t_2 is the ratio between the length from $\vec{W}_b$ to the intersection point and the length of the body edge. In order to have a collision, it is necessary that this intersection point is projected over both edges, which happens only if the ratio is in $[0, 1]$ for both t_1 and t_2 .

This process is done for each one of the 3 edges of the candidate mesh triangle. If a collision is detected the intersection point is stored as a contact point and $\beta_e \vec{n}_e$ is stored as a contact normal, where $\beta_e = \text{sign}(\vec{n}_e \cdot (\vec{X}_b - (t_2 \vec{e}_b + \vec{W}_b)))$ (being $\vec{X}_b$ again the center of the body) for the same reason as in the vertex-face collision case.

- **Momentum update**

Let $\vec{r}$ be the vector between the average of all computed contact points and the center of the body, $\vec{n}_f$ the normalized vector of the sum of all contact normals, m the body mass, $\vec{p}$ its linear momentum, $\vec{\omega}$ its angular velocity, $\mathbf{I}$ its inertia momentum matrix and ξ a restitution coefficient. In case at least one collision is detected, the resulting impulse $\vec{j}$ is given by:

$$\begin{aligned} \vec{j} &= \begin{cases} \frac{-(1+\xi)v_n}{\frac{1}{m} + \vec{n}_f \cdot (\mathbf{I}^{-1}(\vec{r} \times \vec{n}_f) \times \vec{r})} & \text{if } v_n < 0 \\ 0 & \text{if } v_n \geq 0 \end{cases} \\ & : v_n = \frac{1}{m} \vec{p} + \vec{\omega} \times \vec{r} \end{aligned} \quad (7)$$

Finally, the linear momentum of the body is increased by $\vec{j}$ and the angular momentum is increased by $\vec{r} \times \vec{j}$. Both are multiplied by $1 - \gamma$, in which γ is a damping parameter to simulate friction.

4 PROGRAM DESCRIPTION

The program has 4 customized structures/classes: Lights, Meshes, Materials and Graphical Objects. The Tables 1, 2, 3, 4, 5 and 6 summarize the main attributes and methods of each structure that can be used to generate many different graphical objects. Any setup can be easily written inside `configScene()` (in the `config.cpp` file).

Initializers	
Method	Description
create_light	Creates a Light object. Can receive two optional arguments: the width and length of the shadow resolution created by this light
create_mesh	Creates a Mesh object. Must receive a name to identify the mesh and can receive two optional arguments: the path of a mesh file to be loaded (in .off format) and a boolean value to activate or not Loop Subdivision for this mesh.
create_material	Creates a Material object. Must receive a name to identify the material and can receive two optional arguments: the path of a albedo texture and a path of a normal texture.
create_graphical_object	Creates a Graphical Object. Must receive a name to identify the object, a reference to a Mesh and a reference to a Material. Can receive an optional argument that enables or not its physical animation.

Table 1. Initializers

Light main attributes			
Attribute	Type	Default Value	Description
position	glm::vec3	(0.0f, 0.0f, 0.0f)	Sets the position of the light source
color	glm::vec3	(1.0f, 1.0f, 1.0f)	Sets the color of the light source in the normalized RGB space
intensity	float	1.0f	Sets the intensity of the light color
perspec_angle	float	0.0f	Sets the FOV angle of perspective projection to generate shadows. If zero, then it uses ortographic projection.

Table 2. Light main attributes

Mesh main methods	
Method	Description
addPlane	Gives a square plane shape to the Mesh. Can receive one optional argument to set the length of the square's size.
addBox	Gives a block shape to the Mesh. Must receive the width, length and depth of the block and can receive one optional argument to set the block's center in the model space.
addSphere	Gives a spherical shape to the Mesh. Must receive a radius value and can receive one optional argument to define the resolution of the sphere (how many vertices per meridian).
meshFromMatrix	Gives a custom shape to the Mesh, based on a 2D binary matrix, as if each 1 value corresponded to a voxel. Must receive the matrix and can receive as optional argument the dimensions of the each voxel.

Table 3. Mesh main methods

Material main attributes			
Attribute	Type	Default Value	Description
albedo	glm::vec3	(0.0f, 0.0f, 0.0f)	Sets the albedo color of the Material in the normalized RGB space.
specK	float	0.5f	Sets the k coefficient for specular lightining effects on that material.
specA	float	30.0f	Sets the α coefficient for specular lightining effects on that material.
restitution	float	1.0f	Sets the restitution coefficient during collisions with this Material.
damping	float	0.0f	Sets the damping parameter during collisions with this Material.

Table 4. Material main attributes

5 RESULTS IN IMAGES

The following images will show some of the features developed for each module.

5.1 Geometry

Figure 1 displays the 3 basic meshes: the plane, the box and the sphere, while in Figure 2 is given an example of loaded mesh (loading

Graphical Objects main attributes			
Attribute	Type	Default Value	Description
mesh	shared_ptr<Mesh>	nullptr	Access the Graphical Object's Mesh
material	shared_ptr<Material>	nullptr	Access the Graphical Object's Material
rigidAtt	shared_ptr<BodyAttributes>	nullptr	Access the Graphical Object's Body Attributes, if it has.
modelMat	glm::mat4	(1.0)	Access the Graphical Object's world space matrix.

Table 5. Graphical Objects main attributes

Graphical Objects main methods	
Method	Description
setBodyCenter	Changes the physical center of the body (which then updates its modelMat attribute). Must receive the new center.
addBoxBody	Turns the object into a body with a physical shape of a block. Must receive the dimensions of the block
addBoxedBody	Turns the object into a body with a block's moment of inertia while considering as vertices and edges the vertices and edges of a Mesh. Must receive a reference to the Mesh and the dimensions of the block.
addRoundBody	Turns the object into a body with a sphere's moment of inertia while considering as vertices and edges the vertices and edges of a Mesh. Must receive a reference to the Mesh and the radius of the sphere.

Table 6. Graphical Objects main methods

the monkey.off file) and a custom mesh, shaped as a table, which

originated from the matrix: $\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 \end{bmatrix}$.

Note that the first row rendered one single block, due to the algorithm implemented to find rectangles inside the binary matrix. Figure 3 shows the loaded monkey.off after one iteration of the Loop Subdivision implemented.

5.2 Rendering

Figure 4 presents a sphere mesh using a loaded earth.jpg texture and the differences in the specular effect when changing its parameters: In the top image, k is higher, so the white color is more intense than in the bottom image, while α is smaller, so the effect is spread in a wider region than in the bottom image, where it is more concentrated.

Variation of parameters can also be seen in Figure 5. While in the first image there is only a single light of white color, the second image has two lights with different source positions and different colors. In the third image the intensity of the red light was decreased and the shadow projection of the blue light changed from orthographic to a 90° FOV of perspective.

The Figure 6 shows the rendering problems treated: the acne effect (corrected using the bias) in the upper image, and the wrong normal effect on a rotated normal texture (solved with TBN transformation) in the middle image. The bottom image presents the final result with both corrections.

5.3 Simulation

In Figure 7 there are two moments of the vertex collision scenario: the moment of impact between the die and the polyhedron shape

(loaded by the sphere.off file), and the moment of the distancing, without the meshes crossing each other. The impact between the vertex of the polyhedron and the cube can be more clearly seen in the middle image with the wires of the meshes. Note that at the same time both objects are colliding with the table object below, remaining static in the vertical direction.

On the other hand, Figure 8 shows an important case of edge-edge collision: the moment an object falls from a border. The die can only remain in this intermediate position because its edges are colliding with the edges of the surface below, preventing the object to rotate inside the floor at the moment one of its vertices loses contact with the ground.

Figure 9 exemplifies more collision situations that can happen when custom objects interact with each other. The collision that happened in the middle image, for example, produced a torque on the left object making it rotate around its center of mass in a clockwise direction.

Finally, Figure 10 presents the effect of the restitution coefficient ξ during collisions, such that in the first image, with $\xi = 0.8$ the cube achieves a much higher position after colliding with the ground than in the second image, where $\xi = 0.3$.

6 CURRENT LIMITATIONS AND FURTHER IMPROVEMENTS

6.1 Custom mesh

The current algorithm is not able to find the global minimum number of rectangles on the matrix, so it can generate more boxes than needed. Greedy Meshing [5] could be implemented in order to find the minimum amount of blocks necessary to be rendered. Another improvement would be to properly connect the vertices of the mesh,

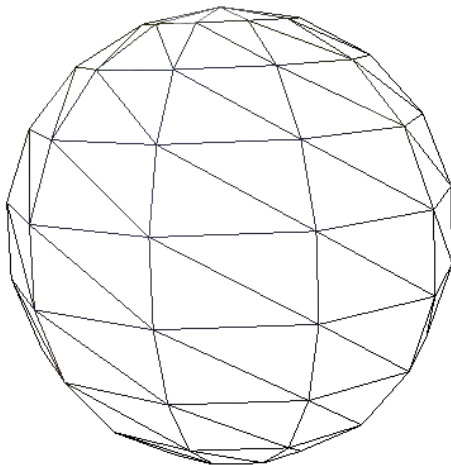
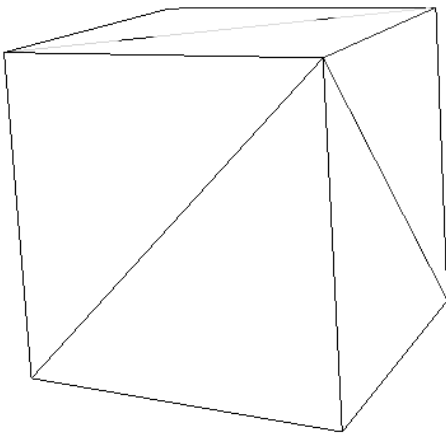
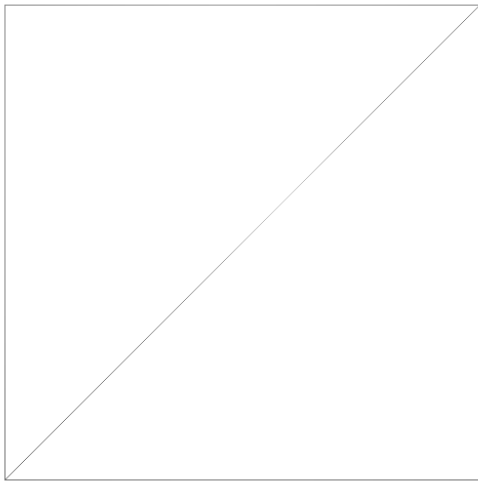


Fig. 1. Plane, Box and Sphere mesh, respectively.

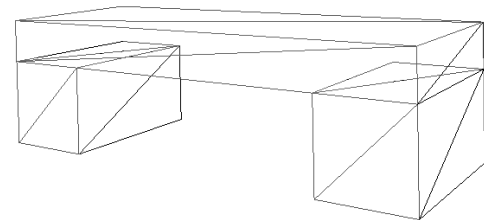
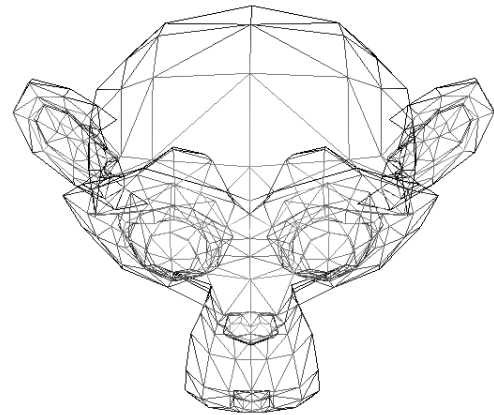


Fig. 2. Loaded monkey.off mesh and custom table drawn in a 2D matrix.

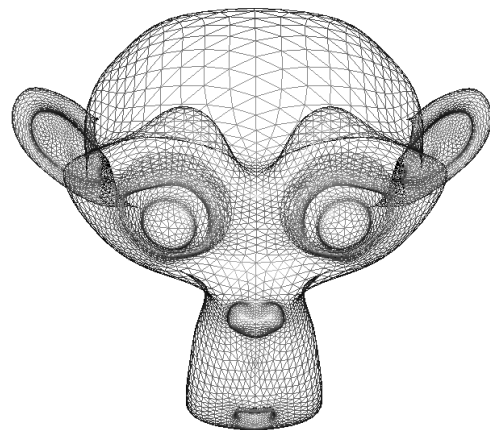


Fig. 3. monkey.off mesh after Loop Subdivision.

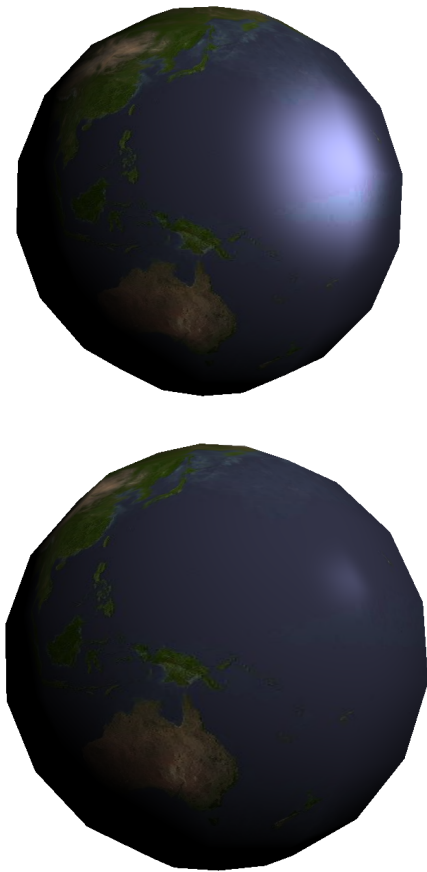


Fig. 4. Changing specular light parameters

which does not happen between different boxes, thus allowing subdivision to work on this kind of mesh.

6.2 Subdivision surface

All implemented meshes have at least one vertex that is attributed at the same time to different indices, which creates a repetition for that vertex in different triangles and consequently causes cracks on the mesh when Loop subdivision is applied. In order to avoid that, it could be guaranteed that one same vertex is linked to one single index in the whole mesh.

6.3 Rigid Body and Collision Solver

The collision algorithm lacks some optimization treatments. Firstly, if two bodies collide there is no need to solve collisions twice, one for each body, because the contact point and the normal vector should be the same (the normals are just opposed). For that reason, the algorithm could be applied to just the pair combinations between bodies, decreasing the amount of computational steps. Secondly, instead of recalculating the triangle normals each time, these vectors could be previously stored and then transformed to world space by just applying the world space matrix of the object. Also, the

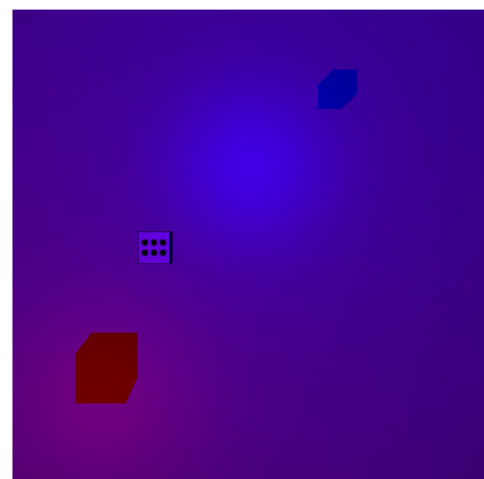
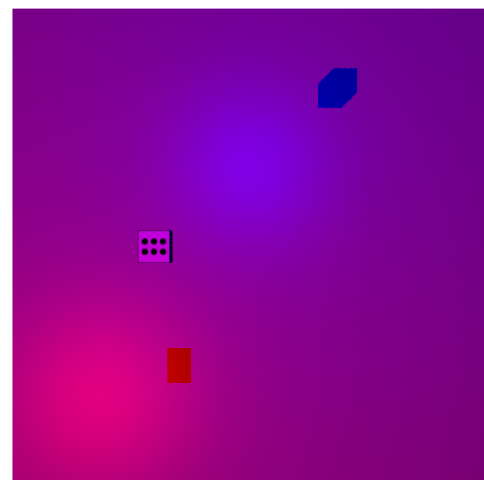
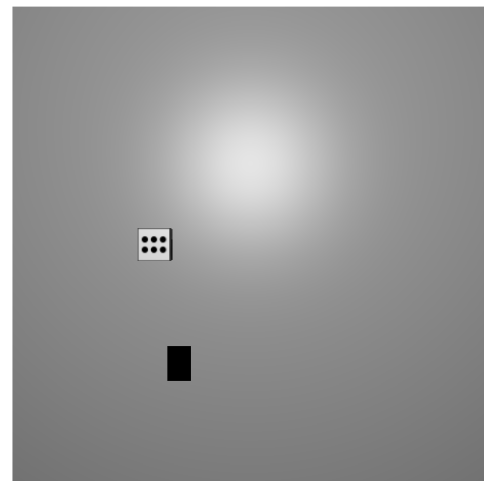


Fig. 5. Changing light attributes.



Fig. 6. Acne effect in the top image, incorrect normal mapping in the middle image and corrected rendering in the bottom image.

conversion from a mesh to a rigid body (as either `BoxedBody` or `RoundBody`) currently stores all vertices of the mesh as vertices of the body (even if there are repeated vertices) and all triangle edges as edges of the body. This is inefficient, as much of this data might not be relevant for collision calculation (points on the inside of the mesh or edges inside faces, for example), so avoiding that would also reduce the amount of computational steps, reducing the lag when many vertices are rendered on the program. Besides, because the collision is not done in pairs, there is a collision case not being correctly treated: when the vertex of body A collides with a face of body B, A would detect this collision but B not necessarily.

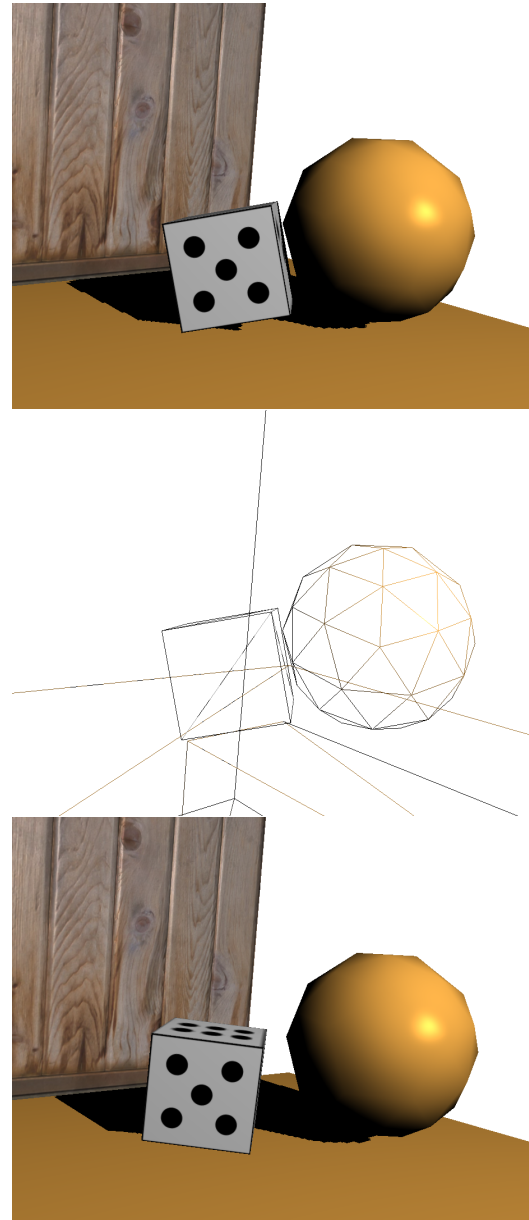


Fig. 7. Vertex collisions.

6.4 API and User Interface

The resulting project creates a set of functions that allows to quickly create and manipulate implemented objects, which could be used as the basis for developing an actual API of Graphical Objects. However, a feature clearly lacking in this project is a proper User Interface to allow a better interaction with such objects. Adding widgets with Qt [8], for example, would be a convenient and helpful contribution to the project, turning the program into an OpenGL platform where users could easily simulate physical interactions between custom rigid 3D models through a comfortable GUI.

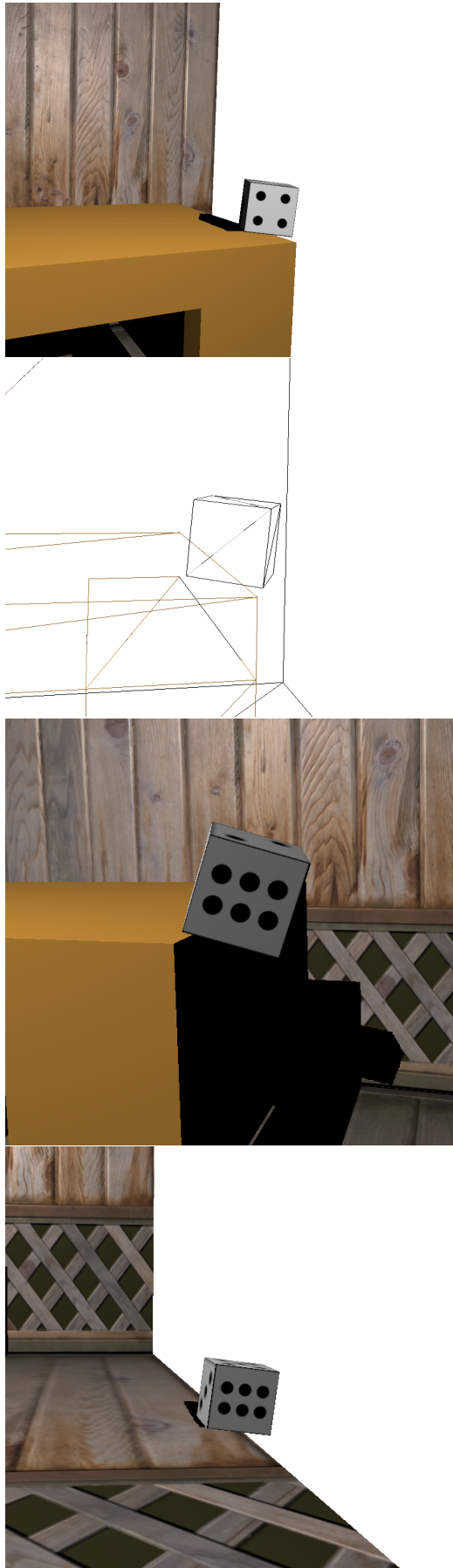


Fig. 8. Edge-Edge collisions.



Fig. 9. Collisions between custom meshes.



Fig. 10. Highest height achieved after collision with restitution coefficient equal to 0.8 on the top image and 0.3 on the bottom image.

7 CONCLUSION

What motivated this project was to unite many of the graphical processing techniques studied during the course in a single platform, inspired by game engines. The resulting program allows many different graphical objects to be rendered in different positions, with different sizes, sharing different meshes, materials and physical bodies.

The most challenging part was to develop the Collision Solver, once each different mesh tested could reveal a new specific interaction that was not being handled correctly, which would require a new revision of the whole algorithm, analyzing in more detail the theoretical physical behavior of the objects on scene in order to discover a better way to adapt the laws of the ideal continuous world to the irregular and discrete digital world of the simulation. Many concepts from spatial geometry were put into practice and helped to deal with problems that were not simple to visualize. Although many changes should still be implemented in order to reduce the unnecessary computations that slow down the execution, the developed Solver was able to work well with a high variety of shapes, including those that could be loaded as external files or created directly with the custom mesh option.

This custom mesh was also a challenge to develop, because the transition from a 2D binary shape to a centralized 3D block shape is not so straightforward. The most difficult task was to create the algorithm capable of finding a smaller set of rectangles inside the

binary matrix to reduce the amount of blocks in the final mesh, once that even thinking on an efficient theoretical algorithm that could guarantee the global minimum is not simple.

This final project for the IG03 - Fundamentals of Computer Graphics course was an opportunity to explore many interesting concepts from graphical rendering, geometry, physics and software development, highlighting that the computer graphics domain is a wide field that incorporates many areas of knowledge in order to build a digital world that can be visualized and interacted with, which is an excellent stage for the human imagination to simulate many different scenarios and create many different realities.

REFERENCES

- [1] Joey de Vries. 2026. Normal Mapping. <https://learnopengl.com/Advanced-Lighting/Normal-Mapping>.
- [2] Epic Games. 2026. Unreal Engine. <https://www.unrealengine.com>.
- [3] Godot Engine. 2026. Godot Engine. <https://godotengine.org>.
- [4] Charles Loop. 1987. Smooth subdivision surfaces based on triangles. (1987).
- [5] Mikola Lysenko. 2012. Meshing in a Minecraft Game. <https://0fps.net/2012/06/30/meshing-in-a-minecraft-game/>.
- [6] Mojang Studios. 2026. Minecraft. <https://www.minecraft.net>.
- [7] Scratchapixel. 2026. Ray-Plane and Ray-Disk Intersection. <https://www.scratchapixel.com/lessons/3d-basic-rendering/minimal-ray-tracer-rendering-simple-shapes/ray-plane-and-ray-disk-intersection.html>.
- [8] The Qt Company. 2026. Qt Development Tools. <https://www.qt.io/development/tools>.
- [9] Unity Technologies. 2026. Unity Game Engine. <https://unity.com/>.
- [10] Valve Developer Community. 2026. Source – Valve Developer Community. <https://developer.valvesoftware.com/wiki/Source>.
- [11] Wikipedia contributors. 2026. Rigid body dynamics — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/wiki/Rigid_body_dynamics.